

The Architect's 20%: Four Adversary Trust Exploitations That Unlock 80% of Attack Patterns

As a security architect with two decades analyzing breaches, I've found a universal truth: **adversaries don't exploit "vulnerabilities"—they exploit *misplaced trust assumptions in system design***. Buffer overflows, phishing, and lateral movement all share the same root cause: systems trusting inputs, identities, or states that should be untrusted. Master these four exploitation patterns, and you'll predict attacker behavior from architecture diagrams—not just incident reports. Each includes a 15-minute lab using only public breach data and free ATT&CK tools—no malware required.

🔑 Concept 1: Trusting Execution Context — "*Assuming Process Identity = Intent*"

Why it unlocks 80%: Privilege escalation, token theft, and living-off-the-land attacks succeed because systems trust *process identity* without verifying *execution context*. A **SYSTEM** process spawned by malware is still **SYSTEM**—the OS trusts the identity, not the intent. This explains why LSASS dumping works and why AppLocker bypasses succeed.

🔧 15-Minute Lab: *Map Identity vs. Context Trust Gap*

```
# Step 1: Observe Windows process identity inheritance (safe demo)
# PowerShell (non-admin):
Start-Process notepad.exe -Verb RunAs # Triggers UAC prompt → spawns as current user
# Now imagine attacker bypasses UAC via DLL hijacking → spawns cmd.exe as SYSTEM
# Identity: SYSTEM | Context: Malicious → OS trusts identity, ignores context

# Step 2: Trace real breach via ATT&CK Navigator
# Go to https://mitre-attack.github.io/attack-navigator/
# Load "FIN7" campaign → Filter techniques: "Token Impersonation/Theft" (T1528)
# Observe: Attackers steal tokens → spawn processes with *legitimate identity but *malicious context

# Step 3: Quantify the trust gap (free tool)
# Search GitHub Security Lab: "query:lsass dump"
# See 50+ public PoCs exploiting identity/context trust gap
```

Interpretation: OS security models trust *who* is executing (user/process identity) but not *why* or *how* execution started (context). Attackers bridge this gap by hijacking legitimate execution paths.

Architectural implication: Design **context-aware execution policies**:

- Windows: Enable Protected Process Light (PPL) for LSASS; use WDAC to block unsigned code execution
- Linux: Deploy seccomp-bpf to restrict syscalls per process context (not just user)
- Cloud: Enforce workload identity with short-lived tokens + runtime attestation (SPIFFE/SPIRE)

 **Underexplored perspective most architects miss:**

The identity/context trust gap creates **asymmetric detection challenges** where defenders see *legitimate identities* performing malicious actions. Example: Defender ATP alerts on "suspicious PowerShell" but misses `rundll32.exe` loading malicious DLL because identity is trusted. Architects who focus on *blocking malicious binaries* miss that **attackers increasingly abuse trusted binaries with malicious contexts** (LOLBAS). The real control isn't signature-based blocking—it's *behavioral baselining*: establish normal execution context patterns per identity (e.g., "lsass.exe never loads from C:\Temp") and alert on deviations.

Concept 2: Trusting State Continuity — "Assuming System State Persists AuthZ Decisions"

Why it unlocks 80%: Lateral movement and persistence succeed because systems make authorization decisions *once* and trust state continuity thereafter. A compromised host remains "trusted" until reboot—even after credential theft. This explains why Golden Ticket attacks work and why session fixation bypasses MFA.

15-Minute Lab: Witness State Continuity Exploitation

```
# Step 1: Map Windows ticket-granting ticket (TGT) lifecycle
# PowerShell (requires domain join—use free-tier Azure AD demo):
klist tickets # Shows Kerberos tickets with 10-hour lifetime
# Attacker steals TGT → Forges Golden Ticket valid for 10 years (Mimikatz)
# → System trusts state continuity: "Valid ticket = authorized user"

# Step 2: Trace real breach via ATT&CK
# ATT&CK Navigator → Load "APT29" campaign → Filter: "Valid Accounts" (T1078)
# Observe: Attackers used stolen credentials for 14 months—system never re-verified trust

# Step 3: Quantify state continuity risk (free tool)
```

```
# Search CISA KEV catalog: https://www.cisa.gov/known-exploited-vulnerabilities-catalog
# Filter "Persistence" → See 30+ exploits abusing state continuity (scheduled tasks, WMI)
```

Interpretation: AuthZ decisions are *point-in-time* events, but systems trust their outcomes *continuously*. Attackers exploit the gap between decision time and execution time.

Architectural implication: Design **continuous re-authentication**:

- Shorten credential lifetimes (Kerberos TGT: 1 hour vs. 10 hours)
- Implement step-up auth for sensitive actions (Azure Conditional Access)
- Deploy host attestation (TPM quotes) to verify system integrity before granting access

 **Underexplored perspective most architects miss:**

State continuity trust creates **temporal attack windows** that scale with credential lifetime. Example: 10-hour Kerberos TGT → attacker has 10 hours to move laterally after initial compromise. Architects who focus on *preventing initial access* miss that **lateral movement speed is determined by state continuity duration**. The real metric isn't "time to detect breach"—it's **mean time to credential expiration (MTTCE)**. Mature programs measure: "What's the maximum time an attacker can operate after credential theft before forced re-authentication?" (Answer should be <15 minutes.)

Concept 3: Trusting Input Origin — "Assuming Network Location = Trustworthiness"

Why it unlocks 80%: SSRF, cloud metadata service abuse, and proxy pivoting succeed because systems trust *network origin* without verifying *request intent*. `169.254.169.254` (AWS metadata) is "local" → trusted by EC2 instances → attackers abuse SSRF to steal credentials. This explains why perimeter security fails in cloud environments.

15-Minute Lab: Map Origin Trust Exploitation

```
# Step 1: Witness SSRF via public demo (safe target)
curl "https://httpbin.org/base64/MTY5LjI1NC4xNjkuMjU0" # Decodes to 169.254.169.254
# Now imagine vulnerable app: curl "https://vulnerable.app/fetch?url=http://169.254.169.254"
# → App trusts "local" origin → leaks AWS keys

# Step 2: Trace real breach via Shodan
# Search: "http.title:'metadata'" port:80
# See exposed cloud metadata endpoints → attackers use SSRF to pivot into cloud accounts

# Step 3: Quantify origin trust failures (free tool)
```

```
# Search GitHub: "filename:main.tf metadata service"
# Find Terraform configs exposing metadata service to containers → SSRF risk
```

Interpretation: Network location (localhost, VPC, on-prem) is treated as a *trust signal*—but attackers manipulate request origin via SSRF, DNS rebinding, or proxy chaining.

Architectural implication: Design **intent-based access control**:

- Never trust network location alone (disable metadata service access by default)
- Require explicit authorization for every resource access (SPIFFE workload identity)
- Deploy egress filtering at workload level (not just network perimeter)

 **Underexplored perspective most architects miss:**

Origin trust failures create **transitive blast radii across trust domains**. Example: Compromise web app in DMZ → SSRF to internal metadata service → steal cloud credentials → own entire cloud account. Architects who segment networks but leave metadata services accessible create *illusory security*. The real control isn't "block SSRF"—it's **trust domain isolation**: metadata services must be accessible *only* via workload identity tokens—not network location. Modern cloud platforms (AWS IMDSv2) enforce this by requiring session tokens—but most orgs disable enforcement for "compatibility."

Concept 4: Trusting Human Judgment — "Assuming Users Detect Social Engineering"

Why it unlocks 80%: Phishing, BEC, and MFA fatigue succeed because systems trust *human verification* without cryptographic proof. Users click links → systems trust their judgment → attackers bypass all technical controls. This explains why 74% of breaches start with human error (Verizon DBIR 2023).

15-Minute Lab: Quantify Human Trust Exploitation

```
# Step 1: Measure phishing susceptibility (safe demo)
# Go to https://haveibeenpwned.com/ → Enter your email
# See if compromised in phishing breaches (e.g., "Collection #1" = credential stuffing)

# Step 2: Trace real breach via CISA alerts
# https://www.cisa.gov/news-events/cybersecurity-advisories/aa23-206a
# Read "Russian Cyber Actors Targeting Critical Infrastructure" → Note: "Initial access"

# Step 3: Quantify human trust gap (free tool)
```

```
# Search OpenPhish: https://openphish.com/feed.txt
# See 5,000+ active phishing URLs daily → Systems trust users to detect them → Failure r
```

Interpretation: Human judgment is treated as a *trust boundary*—but cognitive biases (urgency, authority) make it reliably exploitable. Systems that require human verification without cryptographic fallback assume perfect judgment.

Architectural implication: Design human-proof authentication:

- Phishing-resistant MFA (FIDO2/WebAuthn) — no user judgment required
- Transaction signing with out-of-band approval (not SMS)
- Behavioral biometrics to detect account takeover *after* authN

 Underexplored perspective most architects miss:

Human trust exploitation creates **asymmetric economics** where attacker cost << defender cost. Example: Attacker spends \$50 on phishing kit → compromises \$2M wire transfer. Defenders spend \$500K on security awareness training → reduces click rate from 3% to 2.5%. Architects who treat "user education" as primary control miss that **human judgment cannot be hardened to cryptographic levels**. The real control isn't "better training"—it's *removing humans from trust decisions*: replace password+MFA with passwordless auth (WebAuthn), replace email approvals with cryptographic transaction signing.

Synthesis: How These Concepts Explain Real Breaches

Breach	Exploited Trust Assumption	Architectural Failure
SolarWinds SUNBURST	Execution context	Build server trusted code signing identity without verifying build context
Colonial Pipeline	State continuity	Compromised password remained valid for lateral movement (no re-auth)
Twilio 2022	Input origin	SSRF via vulnerable app → accessed internal Twilio APIs via "trusted" localhost
Twitter 2020	Human judgment	MFA fatigue attack → users approved fraudulent auth requests

The Architect's Mantra:

"Adversaries don't break cryptography—they break trust assumptions. Your job isn't to prevent all attacks—it's to design systems where trust is minimal, time-bound, and cryptographically verified—not assumed from identity, state, origin, or human judgment."

Your Architectural Foundation: The Four Leverage Points

Trust Assumption Exploited	Manifests As	Architectural Control	What Most Miss
Execution context	Privilege escalation, LOLBAS	Context-aware execution policies (WDAC, seccomp)	Attackers abuse <i>trusted binaries</i> —not malware
State continuity	Lateral movement, persistence	Continuous re-authentication + short credential lifetimes	Lateral movement speed = credential lifetime
Input origin	SSRF, cloud pivoting	Intent-based access (workload identity) vs. location-based	Metadata services create transitive blast radii
Human judgment	Phishing, MFA fatigue	Remove humans from trust decisions (FIDO2, transaction signing)	Human judgment cannot reach cryptographic assurance levels

Your Next Challenge (20 Minutes)

Build an "adversary trust exploitation map" for one recent breach:

1. Pick a breach from CISA KEV catalog or MITRE ATT&CK campaigns

2. Trace the trust assumptions exploited:
 - What identity/context gap enabled initial access?
 - How did state continuity enable lateral movement?
 - What origin trust allowed pivoting?
 - Where did human judgment fail?
3. Identify **one architectural control** that would have contained blast radius (e.g., "Shorten Kerberos TGT lifetime to 15 minutes would have limited lateral movement window after initial compromise").

Deliverable: A 3-sentence architectural decision:

"I will implement [control] to break the [identity/context/state/origin/human] trust assumption exploited in [breach] because [risk], reducing blast radius from [entire environment] to [single workload]."

Why This Is the True 20%

These four concepts explain:

- Why privilege escalation works despite "patched systems" (execution context)
- Why lateral movement is fast despite "segmentation" (state continuity)
- Why cloud breaches cascade despite "firewalls" (input origin)
- Why phishing succeeds despite "MFA" (human judgment)

Master these, and you'll design architectures that **fail safely** when trust assumptions break—not by preventing every attack, but by containing blast radius through *minimal, time-bound, cryptographically verified trust*. That's the architect's advantage: understanding that **security isn't about trust—it's about distrust engineered into system design**.